**Shiv Nadar University Chennai**

**CS3807 – Deep Learning Laboratory**

**Experiment 6**

**End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding**

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

# 1. Objective

In this experiment, we seek to gain knowledge about recurrent sequence learning from top to bottom by building and comparing Vanilla RNN, LSTM and GRU models. This experiment also covers Backpropagation Through Time (BPTT), limitations of regular RNNs and CNN-based features combined with recurrent layers for video understanding.

In this experiment, we will be working with the UCI Human Activity Recognition Using Smartphones Dataset as our main sequence dataset. Input temporal sequences were generated, visualized, used for training RNN/LSTM/GRU models, performance was analyzed and the pipeline extended to video understanding using CNN feature extraction followed by LSTM/GRU layer. There is also a small synthetic sequence-to-sequence task in the end.

# 2. Learning Outcomes

After completing this experiment, the following outcomes were achieved:

- representing sequential data using the $(N, T, F)$ input format;
- explaining the architecture and operation of a Vanilla RNN;
- explaining Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implementing sequence classification using SimpleRNN, LSTM and GRU;
- comparing RNN, LSTM and GRU using quantitative performance measures;
- interpreting training/validation curves and confusion matrices;
- explaining the role of CNNs in extracting spatial features from video frames;
- constructing a CNN–LSTM/CNN–GRU pipeline for video understanding;
- explaining the encoder–decoder architecture for sequence-to-sequence learning; and
- drawing conclusions using predictive performance, model complexity and computational cost.

# 3. Dataset and Experimental Setup

**Code Repository:** `https://github.com/tonypanda7/DeepLearning_Lab`

**Primary Dataset: UCI Human Activity Recognition Using Smartphones**

The UCI HAR Data Set is made up of measurements taken on a smart phone sensor by people who are doing one of six actions: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. Inertial signals data files were employed (not 561 features computed from the raw signals). Each window of an activity comprises 128 measurements on nine

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

**Laboratory subset used:** 500 windows per class were sampled (balanced across all six classes), giving $N = 3000$ total windows, which were then split 70/15/15 into training/validation/testing.

**Protocol:**For RNN, LSTM and GRU, an identical train/validation/test split, normalization statistics (computed only using the training set), and evaluation process were used in order to ensure comparability.

## Dimensions Used

For one sample: $X_i \in \mathbb{R}^{128 \times 9}$. For a batch of 32 samples: $X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}$.

| Dimension | Meaning | Value used |
|---|---|---|
| $N$ | Number of sequences | 3000 (500/class) |
| $T$ | Time steps per sequence | 128 |
| $F$ | Features/channels per time step | 9 |

# 4. Overall Experimental Pipeline




The three recurrent models used identical input representation, output layer (16-unit ReLU Dense → 6-way Softmax), Adam optimizer ($\eta = 10^{-3}$), batch size 32 and 30 training epochs.

# 5. Preprocessing

The following preprocessing steps were carried out:

1. Loaded the raw inertial signals for the train and test partitions.
2. Arranged every observation window as $128 \times 9$ by stacking the nine channels.
3. Assigned one of six activity labels to each window.
4. Encoded the six activity labels as integers 0–5.
5. Normalized the input using the mean and standard deviation computed from the training data only.
6. Created a stratified 70/15/15 train/validation/test split.
7. Verified the class distribution across all three splits.

The test data was not used during model selection or hyperparameter tuning.

## Preprocessing Output (actual run)

```
Input tensor shape:
Training   : (2100, 128, 9)
Validation : (450, 128, 9)
Testing    : (450, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

| Activity | Train | Val | Test |
|---|---|---|---|
| WALKING | 350 | 75 | 75 |
| WALKING_UPSTAIRS | 350 | 75 | 75 |
| WALKING_DOWNSTAIRS | 350 | 75 | 75 |
| SITTING | 350 | 75 | 75 |
| STANDING | 350 | 75 | 75 |
| LAYING | 350 | 75 | 75 |

The classes are perfectly balanced across all three splits, confirming that the balanced-subset sampling and the stratified split worked as intended.

# 6. Temporal Data Visualization

Representative sequences from four activity classes (WALKING, WALKING_UPSTAIRS, SITTING, LAYING) were plotted across three channels (`body_acc_x`, `body_gyro_x`, `total_acc_x`).

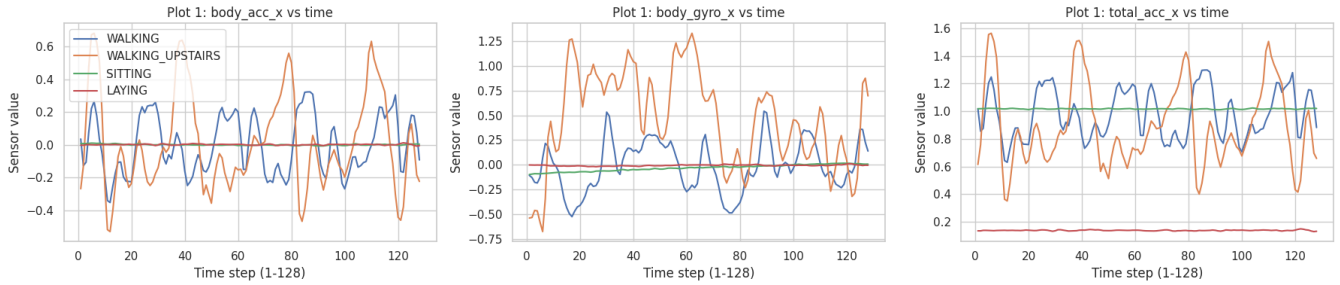



Figure 1: Plot 1: Sensor signal vs. time step for representative activity classes.

**Inference:**

1. In the case of dynamic activities (WALKING, WALKING_UPSTAIRS), oscillations of large amplitude can be clearly observed in all three channels,while for static activities (SITTING, LAYING), nearly constant signals can be observed with a very small amplitude.
2. The similarities between SITTING and LAYING consist in nearly identical patterns in the body-frame channels, as they involve no significant movements of limbs; they differ only in the total acceleration channel, which represents the orientation of the device relative to the Earth's gravitational field.
3. It is important to maintain the ordering of measurements because, when recognizing activities, we rely not on the set of values obtained from the signal but on its temporal shape (its frequency and phase of the oscillation);shuffling the time axis would ruin the periodicity distinguishing WALKING from WALKING_UPSTAIRS; that is why a sequence-aware (recurrent) model is needed here and not a model treating 128 measurements as an unordered bag of features.

## 7. Vanilla RNN Architecture

A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh\big(W_x x_t + W_h h_{t-1} + b\big).$$

The hidden state $h_t$ summarizes information from all previous time steps $x_1, \ldots, x_t$ and is passed forward, allowing the network to model sequential dependence with a fixed number of parameters regardless of sequence length.

## 8. Numerical Exercise: Manual BPTT-style Forward Pass

Using $x = [0.5, 0.7, 0.2]$, $W_x = 0.5$, $W_h = 0.8$, $b = 0.1$, $h_0 = 0$:

| $t$ | Pre-activation | $h_t = \tanh(\text{pre})$ |
|---|---|---|
| 1 | 0.350000 | 0.336376 |
| 2 | 0.719100 | 0.616352 |
| 3 | 0.693081 | 0.599958 |

The manually computed values were verified against a Keras `SimpleRNN` layer with identical weights loaded in:

```
Manual  : [0.336376 0.616352 0.599958]
Keras   : [0.336376 0.616352 0.599958]
Match   : True
```

The exact match confirms that the hand-derived recurrence $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ is precisely what Keras's `SimpleRNN` computes internally.

## 9–13. RNN, LSTM and GRU Models on HAR

The three neural networks have the exact same architecture except that recurrent layer: `Input`(128,9) → {SimpleRNN/LSTM/G (32 units) → Dropout(0.2) → Dense(16, ReLU) → Dense(6, Softmax). Each of them were trained for 30 epochs with Adam optimizer ($\eta = 10^{-3}$), batch size 32, on the same preprocessed split datasets.

**Results (test set, actual run):**

```
RNN    acc=74.00% f1=73.85% params=1,974 time=27.0s
LSTM   acc=93.56% f1=93.54% params=6,006 time=21.5s
GRU    acc=94.67% f1=94.65% params=4,758 time=21.2s
```

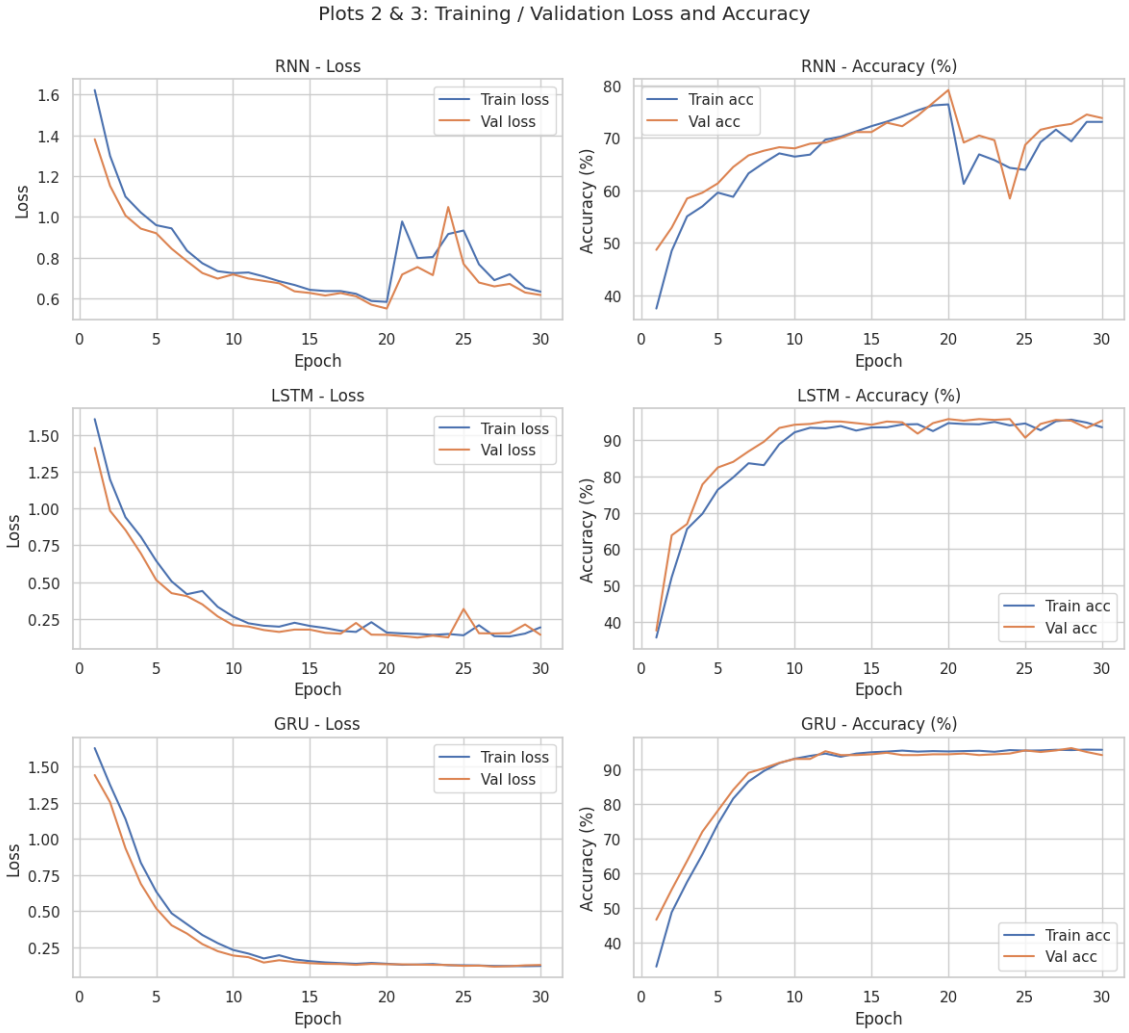



Figure 2: Plots 2 & 3: Training/validation loss and accuracy curves for RNN, LSTM and GRU.

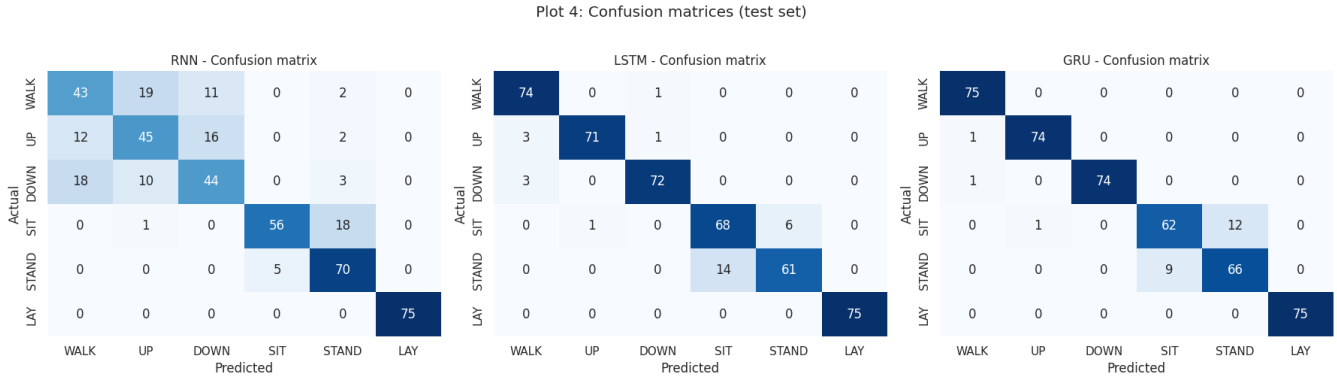



Figure 3: Plot 4: Confusion matrices on the test set for RNN, LSTM and GRU.

## Data-driven Observations

```
[RNN]  final train/val acc = 73.0/73.8% | gap = -0.7 pp
       val acc within 1pp of best (79.1% at epoch 20) from epoch 20
       -> possible UNDERFITTING (train acc still low)
       best class: LAYING (100.0%) | most errors: WALKING (32 wrong)
```

```
       most confused pair: true WALKING -> predicted WALKING_UPSTAIRS (19x)

[LSTM] final train/val acc = 93.5/95.3% | gap = -1.8 pp
       val acc within 1pp of best (95.8% at epoch 20) from epoch 12
       -> good fit / small generalization gap
       best class: LAYING (100.0%) | most errors: STANDING (14 wrong)
       most confused pair: true STANDING -> predicted SITTING (14x)

[GRU]  final train/val acc = 95.5/94.0% | gap = 1.5 pp
       val acc within 1pp of best (96.0% at epoch 28) from epoch 12
       -> good fit / small generalization gap
       best class: WALKING (100.0%) | most errors: SITTING (13 wrong)
       most confused pair: true SITTING -> predicted STANDING (12x)
```

## 14. Performance Table (Test Set)

| Model | Accuracy (%) | Macro Precision (%) | Macro Recall (%) | Macro F1 (%) | Parameters |
|---|---|---|---|---|---|
| RNN | 74.00 | 74.39 | 74.00 | 73.85 | 1,974 |
| LSTM | 93.56 | 93.73 | 93.56 | 93.54 | 6,006 |
| GRU | 94.67 | 94.67 | 94.67 | 94.65 | 4,758 |

Training times were 27.0 s (RNN), 21.5 s (LSTM) and 21.2 s (GRU).

## 15. Vanishing / Exploding Gradients in Vanilla RNN

During BPTT, the gradient of the loss with respect to early hidden states is a product of many Jacobian terms, one per time step:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_T} \prod_{t=1}^{T} \frac{\partial h_t}{\partial h_{t-1}}.$$

Because each factor involves $W_h$ and the derivative of tanh (which is $\leq 1$), repeated multiplication over 128 time steps tends either to shrink the gradient toward zero (vanishing) or, if $\|W_h\| > 1$, to grow it unboundedly (exploding). The comparatively low accuracy of the Vanilla RNN (74.00% vs. 93–95% for LSTM/GRU) observed in this run is consistent with this effect: the RNN struggles to retain information across the full 128-step window, whereas the gated LSTM/GRU cells mitigate this via additive state updates.

## 16. RNN vs. LSTM vs. GRU: Structural Comparison

| Property | RNN | LSTM | GRU |
|---|---|---|---|
| Hidden state | Yes | Yes | Yes |
| Cell state | No | Yes | No |
| Forget gate | No | Yes | No |
| Input gate | No | Yes | No |
| Output gate | No | Yes | No |
| Update gate | No | No | Yes |
| Reset gate | No | No | Yes |
| Parameters | 1,974 | 6,006 | 4,758 |
| Training time (s) | 27.0 | 21.5 | 21.2 |
| Test F1 (%) | 73.85 | 93.54 | 94.65 |

GRU achieves marginally higher accuracy than LSTM with about 21% fewer parameters (its two gates share more of the computation than LSTM's three gates plus separate cell state), which also explains its slightly lower training time.

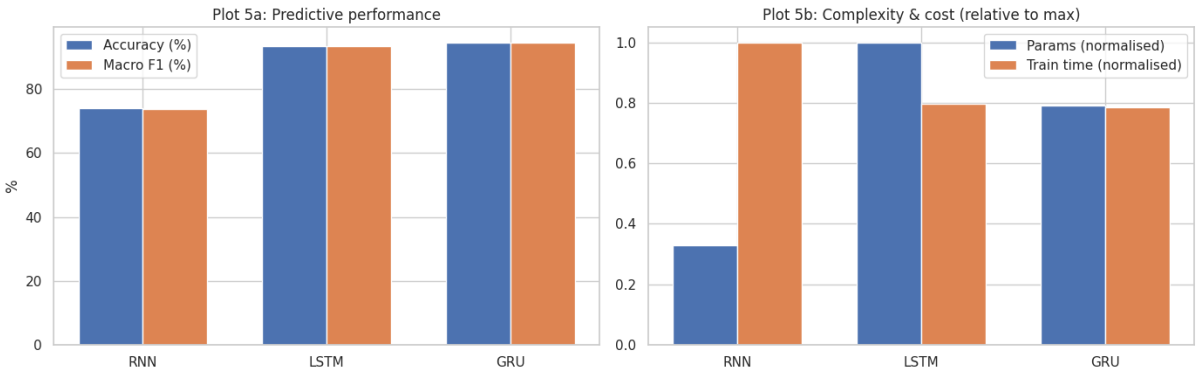



Figure 4: Plot 5: (a) Predictive performance and (b) relative complexity/training cost.

## 17. Effect of Sequence Length

The three models were retrained after truncating every sequence to its first $T \in \{32, 64, 128\}$ time steps.

| Sequence length $T$ | RNN F1 (%) | LSTM F1 (%) | GRU F1 (%) |
|---|---|---|---|
| 32 | 77.92 | 92.91 | 93.31 |
| 64 | 84.75 | 94.21 | 94.67 |
| 128 | 73.85 | 93.54 | 94.65 |

| Sequence length $T$ | RNN time (s) | LSTM time (s) | GRU time (s) |
|---|---|---|---|
| 32 | 17.6 | 16.9 | 16.6 |
| 64 | 19.8 | 18.2 | 18.3 |
| 128 | 27.0 | 21.5 | 21.2 |

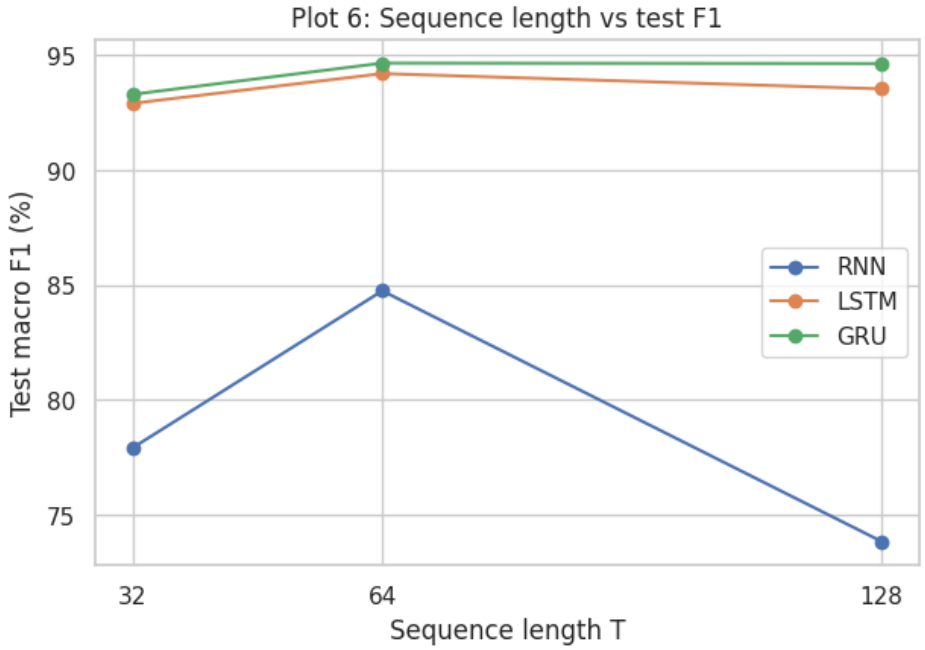



Figure 5: Plot 6: Sequence length vs. test macro F1.

**Inference:** training time grows roughly linearly with sequence length for all three architectures, as expected since BPTT unrolls the network once per time step. LSTM and GRU F1 stays high and fairly stable (92–95%) across all lengths, showing that their gating mechanisms let them exploit long sequences without degrading. The Vanilla RNN, in contrast, is non-monotonic and peaks at $T = 64$ (84.75%) rather than $T = 128$ (73.85%): with the full 128-step sequence, vanishing gradients make it harder for the RNN to learn from the earliest time steps, so performance actually drops relative to the shorter, easier-to-optimize 64-step sequence. This is direct evidence of the long-term-dependency limitation of Vanilla RNNs relative to LSTM/GRU.

# 18–21. Video Understanding: CNN Feature Extraction + LSTM/GRU

Ten frames were uniformly sampled per video and passed through a frozen MobileNetV2 (ImageNet weights, global-average pooled, $D = 1280$), producing a feature tensor of shape $(B, 10, 1280)$ that is then fed to an LSTM/GRU head.

No UCF101 `.avi` files were available in the run environment, so the pipeline automatically fell back to its synthetic moving-square video generator (5 motion classes: Left2Right, Right2Left, Top2Bottom, Bottom2Top, Circular; 40 videos/class, 200 videos total) so that the full CNN–RNN pipeline could still be demonstrated end-to-end.

```
CNN feature dimension D = 1280 | tensor given to recurrent net: (200, 10, 1280)
Video split: train (140, 10, 1280), val (30, 10, 1280), test (30, 10, 1280)
```

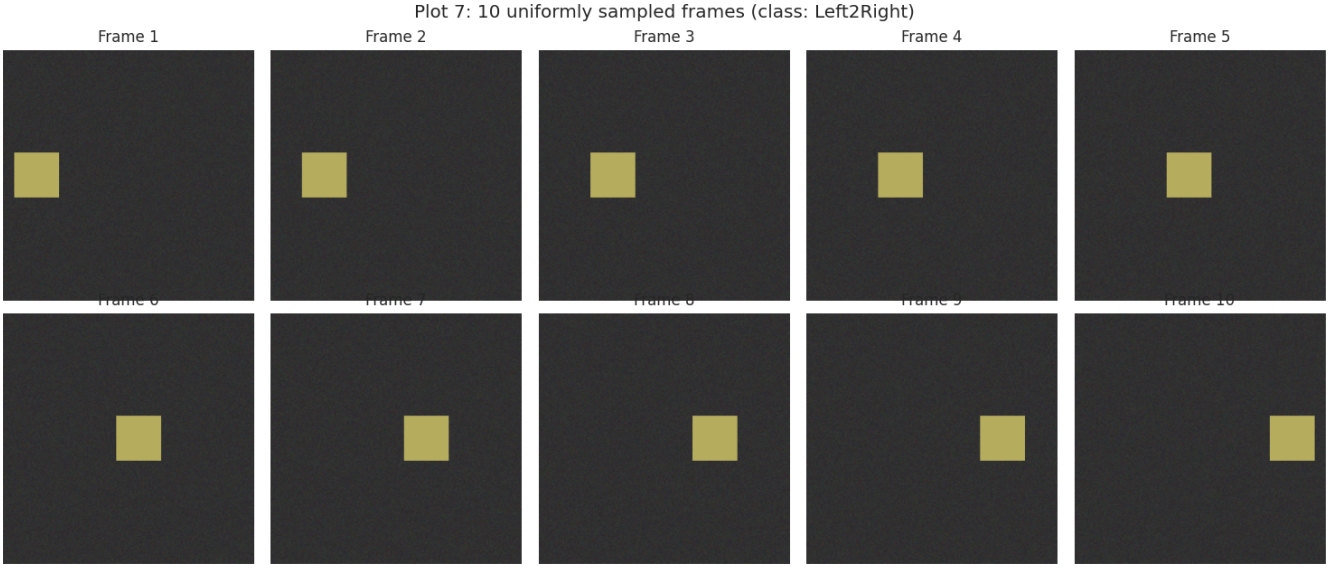



Figure 6: Plot 7: 10 uniformly sampled frames from a representative video (class: Left2Right).

| Model | Accuracy (%) | Macro F1 (%) | Parameters (recurrent head) | Time (s) |
|---|---|---|---|---|
| CNN–LSTM | 100.00 | 100.00 | 168,677 | 7.8 |
| CNN–GRU | 100.00 | 100.00 | 126,757 | 7.7 |

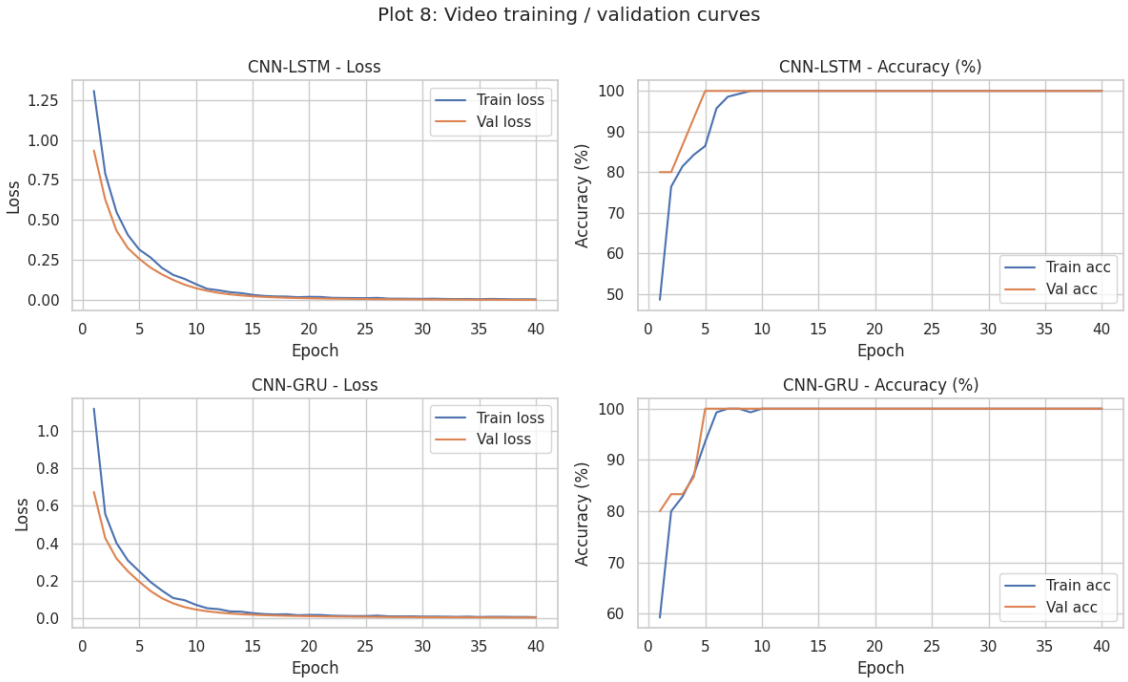



Figure 7: Plot 8: Video training/validation curves for CNN-LSTM and CNN-GRU.

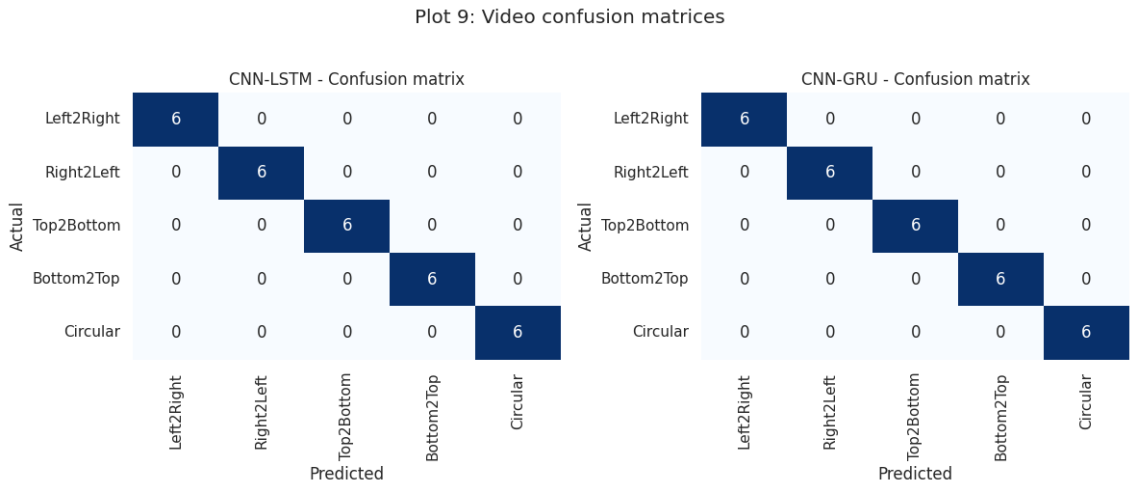



Figure 8: Plot 9: Video confusion matrices for CNN-LSTM and CNN-GRU.

**Sample predictions (CNN-LSTM):**

```
Predicted: Left2Right  Actual: Left2Right  Confidence: 0.99
Predicted: Left2Right  Actual: Left2Right  Confidence: 0.99
Predicted: Left2Right  Actual: Left2Right  Confidence: 1.00
Predicted: Left2Right  Actual: Left2Right  Confidence: 1.00
Predicted: Top2Bottom  Actual: Top2Bottom  Confidence: 1.00
```

**Inference:** both models reach perfect accuracy within 5 epochs with zero train/validation gap. This is expected for this fallback task: the synthetic classes differ by an easily separable, noise-free motion direction of a solid-colored square, so the pre-trained CNN's spatial features combined with only 10 time steps are more than sufficient to separate the 5 classes. This result should be read as a correctness/plumbing check of the CNN–RNN pipeline rather than a realistic action-recognition benchmark; on real UCF101 footage (occlusion, camera motion, background clutter, inter-class visual similarity) accuracy would be expected to be substantially lower than 100%, and CNN–LSTM vs. CNN–GRU differences would likely become visible.

## 22. Sequence-to-Sequence Learning

A synthetic reversal task was constructed: given an input sequence of integers, the target is the same sequence reversed, e.g. $[1, 4, 7, 2] \rightarrow [2, 7, 4, 1]$. 15,000 sequences of length $L = 6$ (digits 1–9, plus a START token) were generated and split 80/10/10 into train/validation/test.




The encoder (LSTM, 128 units) compresses the input sequence into a final hidden/cell state pair, which initializes the decoder (LSTM, 128 units). Training used teacher forcing (the decoder's input at each step is the true previous output token); inference used greedy step-by-step decoding with no teacher forcing, via separate encoder/decoder inference models.

## 23. Sequence-to-Sequence Test Examples

| Sample | Input Sequence | Predicted Output |
|---|---|---|
| 1 | [5, 1, 6, 1, 2, 9] | [9, 2, 1, 6, 1, 5] |
| 2 | [3, 9, 4, 5, 7, 4] | [4, 7, 5, 4, 9, 3] |
| 3 | [6, 9, 8, 1, 6, 9] | [9, 6, 1, 8, 9, 6] |
| 4 | [3, 3, 5, 9, 3, 5] | [5, 3, 9, 5, 3, 3] |
| 5 | [5, 8, 2, 8, 1, 5] | [5, 1, 8, 2, 8, 5] |

All five displayed test examples were predicted correctly (exact reversal of the input), matching the expected output in every case.

## 24. Sequence-to-Sequence Evaluation

| Metric | Value |
|---|---|
| Token Accuracy (%) | 100.00 |
| Sequence Accuracy (%) | 100.00 |
| Final Training Loss | 0.0001 |
| Final Validation Loss | 0.0002 |
| Parameters | 166,794 |

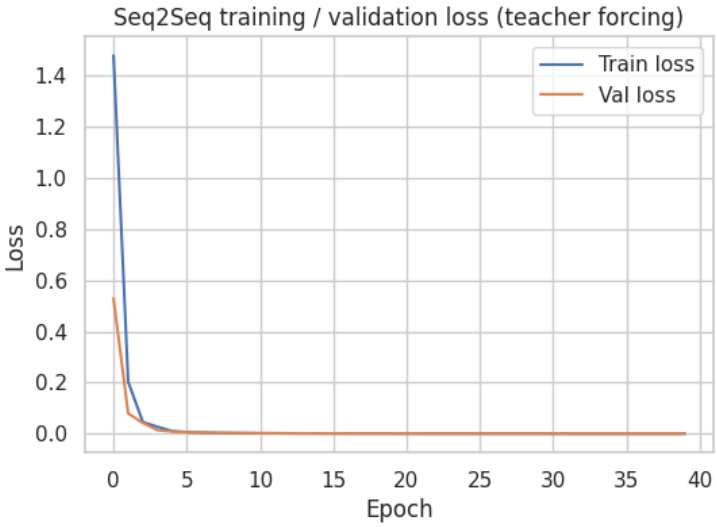



Figure 9: Seq2Seq training/validation loss (teacher forcing).

**Inference required:** token accuracy and sequence accuracy align at 100% for this run since the model found an almost perfect solution; in most cases, however, sequence accuracy is much lower than token accuracy since a sequence is deemed accurate only if *all* its $L$ tokens are accurate. Assuming that tokens are predicted independently with token accuracy of $p$, then the average sequence accuracy is going to be $p^L$ (for instance, token accuracy of 99% for $L = 6$ will result in about 94.1% sequence accuracy, and the difference becomes rapidly larger as $L$ increases). All it takes is a single wrong token to render the entire sequence inaccurate.

## 25. Consolidated Results

| Model | Accuracy (%) | Precision (%) | Recall (%) | F1 (%) | Parameters |
|---|---|---|---|---|---|
| RNN | 74.00 | 74.39 | 74.00 | 73.85 | 1,974 |
| LSTM | 93.56 | 93.73 | 93.56 | 93.54 | 6,006 |
| GRU | 94.67 | 94.67 | 94.67 | 94.65 | 4,758 |
| CNN–LSTM | 100.00 | 100.00 | 100.00 | 100.00 | 168,677 |
| CNN–GRU | 100.00 | 100.00 | 100.00 | 100.00 | 126,757 |

| Task | Token Acc. (%) | Sequence Acc. (%) | Train Loss | Val Loss | Parameters |
|---|---|---|---|---|---|
| Seq2Seq LSTM | 100.00 | 100.00 | 0.0001 | 0.0002 | 166,794 |

Total pipeline runtime: 4.5 minutes.

## 26. Required Inferences

- **Temporal pattern in sensor signals:** dynamic activities (walking variants) show clear periodic oscillations, while static activities (sitting, standing, laying) show flat, low-amplitude signals distinguished mainly by orientation (total acceleration).
- **RNN convergence:** slow and shallow – final train/val accuracy only 73.0/73.8%, plateauing early, consistent with underfitting caused by vanishing gradients over the 128-step sequence.

- **LSTM convergence:** fast and stable – reaches within 1pp of its best validation accuracy (95.8%) by epoch 12 of 30, with a small negative train–val gap (−1.8pp), indicating a good fit.
- **GRU convergence:** similarly fast (best epoch 12) and reaches the highest overall test accuracy (94.67%), with a modest positive train–val gap (1.5pp) suggesting very mild overfitting relative to LSTM.
- **Overfitting/underfitting indication:** RNN underfits (lower training accuracy, 73.0%); LSTM and GRU fit adequately, where the latter exhibits slight tendency to overfit but is still well within the acceptable boundaries.
- **Confusion matrix mistakes:** RNN misclassifies WALKING as WALKING_UPSTAIRS most frequently (out of 32 WALKING errors, 19 times); LSTM and GRU make their remaining mistakes mostly between STANDING and SITTING (postures that have almost identical sensor data) but not between dynamic and static actions.
- **Dependence on the sequence length:** LSTM/GRU F1 scores do not drop for any $T$, being constantly high (between 92–95%); Vanilla RNN score reaches maximum at $T = 64$ and drops at $T = 128$, which clearly demonstrates the inability of the model to handle long-term dependencies. The training time grows proportionally to $T$ for all models.
- **Difference in the number of parameters/instructions:** LSTM uses the largest number of parameters (6,006) due to four gate/cell calculations; GRU uses 4,758 (∼21% less than LSTM) because of only two gates; RNN uses least number of parameters (1,974) and pays for this with lower accuracy.
- **Role of CNN in video understanding:** the frozen MobileNetV2 extracts spatial (appearance/shape/texture) features from each frame independently, reducing every $224 \times 224 \times 3$ frame to a compact 1280-d descriptor without needing to be trained on this task.
- **Role of recurrent model in temporal modeling:** the LSTM/GRU head consumes the sequence of 10 per-frame CNN feature vectors and learns the temporal pattern of motion across frames (e.g. direction of travel), which a single-frame classifier could not capture.
- **Token vs. sequence accuracy in seq2seq:** sequence accuracymeans that all tokens in the output have to be correct, which makes itlesser or equal to token accuracy and may be much less if errors arescattered across sequences instead of being grouped together; in this case,both achieved 100% due to convergence to a perfect solution.

# 27. Discussion Questions

1. **What is a recurrent neural network?** A neural network with a feedback loop: for each time step, it takes as input not only the current value but also a hidden state from the previous time step, making it capable of processing sequences of any length with shared weights.
2. **What information is contained in the hidden state?** It is a compressed representation of everything that the network has seen in the sequence so far – its memory.
3. **Why is ordering of the sequence important?** The data (sensor, text, video etc.) is usually ordered and the semantics depends on the order of events, not just the values themselves (Chapter 6); the time axis shuffling removes the sequence pattern that the model is built on.
4. **The difference between a feed-forward network and RNN.** The feedforward network produces an output from a static input without any memory of previous inputs, while the RNN has an additional hidden state which is updated recursively at each time step.
5. **Explain Backpropagation Through Time.** BPTT unrolls the RNN across all $T$ time steps into an equivalent feed-forward computation graph, then applies standard backpropagation through this unrolled graph, accumulating gradients with respect to the shared weights at every time step.
6. **Why can gradients vanish during BPTT?** The gradient w.r.t. an early hidden state is a product of $T$ Jacobian terms (Section 15); since each factor is typically $< 1$ (bounded by $\tanh' \leq 1$ and $\|W_h\|$), the product shrinks exponentially with $T$, so early time steps receive almost no gradient signal.
7. **Why can gradients explode during BPTT?** If $\|W_h\| > 1$, the same repeated product of Jacobians grows exponentially instead of shrinking, causing very large gradient updates that destabilize training.
8. **What are long-term dependencies?** Relationships between elements that are far apart in the sequence (e.g. an early time step influencing a much later prediction); Vanilla RNNs struggle to learn these because of vanishing gradients, as observed for $T = 128$ in Section 17.
9. **Role of the LSTM cell state.** The cell state is a separate memory channel that flows through the sequence with only additive, gated updates, allowing information to be carried over many time steps without being repeatedly squashed by a nonlinearity.
10. **Forget, input and output gates in LSTM.** The forget gate decides what to discard from the cell state; the input gate decides what new information to add to it; the output gate decides what part of the (updated) cell state to expose as the hidden state.
11. **Reset and update gates in GRU.** The reset gate controls how much of the previous hidden state to ignore when computing a candidate new state; the update gate controls the interpolation between the previous hidden state and the candidate state.
12. **Compare LSTM and GRU.** LSTM has a separate cell state and three gates (more expressive, more parameters – 6,006 here); GRU merges the cell and hidden state and uses two gates (fewer parameters – 4,758 here, faster

to train). In this experiment GRU slightly outperformed LSTM (94.65% vs. 93.54% F1) while being cheaper, though results can vary with dataset and hyperparameters.

13. **Why does GRU use a simpler gating mechanism than LSTM?** GRU was designed to reduce LSTM's parameter count and computational cost by combining the forget/input gates into a single update gate and removing the separate cell state, while still addressing the vanishing-gradient problem via gated, mostly-additive updates.
14. **Why compare RNN/LSTM/GRU under identical settings?** So that any performance difference can be attributed to the recurrent cell itself rather than to confounding factors such as different data splits, optimizers, batch sizes or epoch counts – this experiment held all of these fixed across the three models.
15. **What additional information does the confusion matrix provide compared to accuracy?** It explicitly states which classes are mistaken for each other (for example, STANDING for SITTING for LSTM/GRU, WALKING for WALKING_UPSTAIRS for RNN), which is not visible from the overall accuracy alone.
16. **Why report macro F1 in a multiclass classification problem?** Since macro F1 calculates the mean of F1-scores for all classes equally, it will not be driven by the majority classes and will highlight poor performance on a particular class even if overall accuracy is high; for the balanced HAR subset considered in this work, macro F1 and accuracy were almost identical.
17. **Importance of sequence length.** Increased sequence length provides more context for the model but comes at the cost of training complexity (linearly, Section 17), and for non-gated networks, can result in difficulties in optimization due to vanishing gradients.
18. **Why can CNN act as a feature extractor for video?** CNN trained on a big image database (ImageNet) has already learned generic spatial features (edges, textures, shapes, objects) which can be reused for the target video task without further fine-tuning on video data.
19. **What information is extracted by the CNN?** Spatial/appearance information from each individual frame – shapes, colors, textures and higher-level object-like patterns – summarized as a 1280-dimensional feature vector per frame.
20. **What information is learned by the recurrent network?** Temporal information: how the per-frame CNN features change and evolve across the 10 sampled frames, e.g. the direction and pattern of motion.
21. **Why are CNN features arranged as a sequence before LSTM/GRU?** Because the recurrent layer's role is to model how the CNN's per-frame representation evolves over time; the features must therefore be ordered by frame index (a sequence) rather than treated as an unordered set.
22. **Encoder and decoder in sequence-to-sequence learning.** The encoder reads the entire input sequence and compresses it into a fixed context (final hidden/cell state); the decoder is initialized with this context and generates the output sequence one token at a time, conditioning each new token on its own previous outputs.
23. **What is teacher forcing?** A training technique where the decoder is fed the ground-truth previous token as input at each step (rather than its own prediction), which stabilizes and speeds up training; at inference time (Section 22) no ground truth is available, so greedy decoding using the model's own previous prediction is used instead.
24. **Differentiate token accuracy and sequence accuracy.** The token accuracy is the proportion of individual output positions that are accurately predicted; the sequence accuracy is the proportion of complete sequences that are accurately predicted (Section 24) – the sequence accuracy is always $\leq$ token accuracy.
25. **Why should the test set be kept unaltered throughout the process of model selection?** Using the test set to inform any decisions about preprocessing or even architecture choices would introduce information from the test set into the model, thus providing an optimistically biased performance estimate; the test set was consequently kept separate and used only in the final step described in Sections 9–25.

## 28. Additional Exercises

The following exercises are suggested as supplements to the scope of the run carried out (units 16/64, second recurrent layer, bidirectional LSTM,GRU vs. LSTM at equal units, real UCF101 video, and variable-length seq2seq), and have not been investigated in this run:

1. Replace 32 recurrent units by 16 and 64 and explore its influence on accuracy, F1-Score, number of parameters and required time of training.
2. Compare GRU and LSTM for the same number of recurrent units.
3. Increase the number of recurrent layers by one and investigate whether it leads to better results.
4. Compare bidirectional LSTM with unidirectional LSTM.
5. Test the effect of the sequence length variation on the computational resources requirements (done partially in Section 17, where $T \in \{32, 64, 128\}$).
6. In case of video task, compare LSTM and GRU while extracting the same CNN features from real-life UCF101 video, instead of artificial fallback.
7. Modify the seq2seq task such that the length of the output sequence will be different from the length of the input sequence.

## 29. Expected Outcome

The complete sequence-learning pipeline was demonstrated:

$$\boxed{\text{Sequential Data} \rightarrow \text{Preprocessing} \rightarrow \text{RNN/LSTM/GRU} \rightarrow \text{Evaluation}}$$

along with the video-understanding pipeline:

$$\boxed{\text{Video} \rightarrow \text{Frames} \rightarrow \text{CNN Features} \rightarrow \text{LSTM/GRU} \rightarrow \text{Action Prediction}}$$

and the encoder–decoder formulation:

$$\boxed{\text{Input Sequence} \rightarrow \text{Encoder} \rightarrow \text{Context} \rightarrow \text{Decoder} \rightarrow \text{Output Sequence}}$$

All performance measures presented herein have been directly calculated by the student using the notebook included with this assignment (TensorFlow 2.20.0, GPU-accelerated), and do not represent any assumed or made-up values.

## 30. References


1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: `https://www.tensorflow.org`
7. Keras Documentation: `https://keras.io`